

s5.02 — Distributed object storage — supporting reference

Supporting reference s5.02 in [the bank](#); not a ranked question. Backs s3.06 tiered storage, s1.07 segment upload, s1.09 upload target, and the video-streaming draft.

JD tie-in: OCI Object Storage is a flagship platform service; metadata/data separation + FD/AD placement is native vocabulary.

Index

1. Crux & why hard
2. Clarifying questions
3. FR / NFR
4. Back-of-envelope
5. HLD: metadata plane vs data plane
6. Write path (key code: manifest commit)
7. Read path
8. Placement & repair
9. Data model
10. Durability math & erasure coding
11. Failure modes
12. Trade-offs & defeaters
13. Pop-ups
14. Memorization card

1. Crux & why hard

- Naive: store blobs as rows in the KV/DB → item-size caps, compaction wrecked by MB values, backups impossible; or "one NFS server" → SPOF and one box's disks.
- Minimum primitives: **split metadata (small, hot, consistent) from data (big, dumb, replicated); immutable chunks; write-then-commit manifest** (atomic visibility); **background repair** to hold replication factor.
- 🗣️ "Metadata is a strongly-consistent/J\ database problem; data is a placement-and-repair problem. Never let the two mix."

2. Clarifying questions (assumed answers)

- Object size range? → KB to TB (chunk anything > 64 MB)
- Consistency? → read-after-write for new objects; overwrite = new version
- Durability/availability target? → 11 nines durability, 4 nines availability (drives replication + repair)
- Access pattern? → write-once read-many; no partial in-place updates

3. FR / NFR

Req	Target	Enforced by
PUT/GET	GET p99 < 100 ms first byte	metadata lookup + direct chunk read
Atomic visibility	object appears all-or-nothing	manifest commit last
Durability	survive disk/host/FD loss	N=3 across FDs or EC 8+4
Scale	billions of objects, EB data	shard metadata; add chunk servers linearly
Consistency	read-after-write on commit	metadata is the truth, strongly consistent

4. Back-of-envelope

- 1B objects × ~500 B metadata ≈ 500 GB metadata — one sharded DB tier, cacheable — so what: metadata fits in "normal DB" world; data plane is where exabytes live
- 1 TB object = 16k × 64 MB chunks; parallel upload across chunk servers — so what: chunking is also the parallelism and retry unit (s1.09 reuses this exact pattern client-side)

5. HLD

```
client
| PUT/GET (auth via gateway)
▼
metadata service [control-ish plane, strongly consistent]
  bucket/key → object_version → chunk list → chunk locations
  | issues signed chunk-server URLs
  ▼
chunk servers [data plane: dumb, huge disks]
  store immutable chunks (checksummed), replicate N=3 across FDs
[async] placement/repair service: scrub checksums, re-replicate lost chunks
```

- Data bytes never traverse the metadata service — clients talk to chunk servers directly (that's how GET stays fast and metadata stays small).

6. Write path (key code: manifest commit)

- Upload chunks first (any order, retries fine — they're invisible), then commit the manifest; commit is the single atomic step.

```
def put_object(meta, chunkstore, bucket, key, stream, chunk_mb=64):
    chunks = []
    for i, block in enumerate(read_blocks(stream, chunk_mb * 2**20)):
        cid = sha256(block) # content-addressed id
        targets = meta.pick_targets(cid, n=3) # 3 servers, distinct FDs
        for t in targets:
            chunkstore.put(t, cid, block) # idempotent by cid
        chunks.append({"seq": i, "cid": cid, "size": len(block), "locs": targets})
    # nothing visible yet; now the atomic step:
    return meta.commit(bucket, key, chunks) # new version row, single txn
# crash before commit -> orphan chunks only (GC later); never a half-visible object
```

```
ObjectVersion putObject(Meta meta, ChunkStore store, String bucket, String key,
                        InputStream in) throws IOException {
    List<ChunkRef> chunks = new ArrayList<>();
    int seq = 0;
    for (byte[] block : Blocks.of(in, 64 << 20)) {
        String cid = Hashing.sha256().hashBytes(block).toString();
        List<Node> targets = meta.pickTargets(cid, 3); // distinct fault domains
        for (Node n : targets) store.put(n, cid, block); // idempotent by cid
        chunks.add(new ChunkRef(seq++, cid, block.length, targets));
    }
    return meta.commit(bucket, key, chunks); // one transaction
}
// content-addressing gives idempotent chunk PUTs + free dedup of identical blocks
```

7. Read path

- GET: metadata lookup (cacheable) → chunk list + locations → parallel range reads from nearest replica → verify checksums → stream
- Range GET: manifest maps byte offset → chunk seq (fixed chunk size = simple division)
- Hot object → more replicas for its chunks or CDN in front (immutability makes caching trivial)

8. Placement & repair

- Placement: pick 3 chunk servers in **distinct fault domains** (host/rack/FD; region-level = separate replication policy); balance by free space + load — an explicit allocator beats a pure hash ring when hardware is heterogeneous (s3.04 ring is the alternative to name)
- Repair: chunk servers heartbeat (s1.04 pattern) + report inventory; repair service diffs expected-vs-present → re-replicates under-replicated chunks, prioritized by how many copies remain (1-copy chunks first)
- Scrub: background checksum reads catch bit rot; corrupt copy → replace from a good replica

9. Data model

```
CREATE TABLE objects (  
  bucket      TEXT NOT NULL,  
  key         TEXT NOT NULL,  
  version     BIGINT NOT NULL,  
  size_b      BIGINT NOT NULL,  
  etag        TEXT NOT NULL,          -- hash of chunk cids  
  state       TEXT NOT NULL,          -- COMMITTED | DELETED  
  created_at  TIMESTAMPTZ NOT NULL,  
  PRIMARY KEY (bucket, key, version)  
);  
CREATE TABLE object_chunks (  
  bucket TEXT, key TEXT, version BIGINT,  
  seq    INT NOT NULL,  
  cid    TEXT NOT NULL,              -- sha256 (content-addressed)  
  size_b INT NOT NULL,  
  PRIMARY KEY (bucket, key, version, seq)  
);  
CREATE TABLE chunk_locations (  
  cid  TEXT NOT NULL,  
  node TEXT NOT NULL,  
  state TEXT NOT NULL,              -- PRESENT | REPAIRING  
  PRIMARY KEY (cid, node)  
);  
-- metadata sharded by hash(bucket, key); listings served by (bucket, key) order
```

10. Durability math & erasure coding

- N=3 replication = 3× cost; erasure coding 8+4 = 1.5× cost, survives any 4 lost fragments — but rebuild reads 8 fragments (repair traffic) and small objects don't amortize
- 📌 Policy: hot/small → 3× replicas; cold/large → EC 8+4; that hybrid is the production answer (and the cost question's answer)

11. Failure modes

- Chunk server dies → repair re-replicates its chunks from peers; durability dips are measured in "time at 2 copies" — alarm on repair backlog
- Metadata shard down → its keyspace unavailable (availability hit, not durability); replicate metadata store (Raft/quorum, s3.04)
- Crash mid-upload → orphan chunks; GC: chunks unreferenced by any manifest and older than T are deleted (manifest-commit ordering makes this safe)
- Delete → tombstone version, async chunk deref + GC (never synchronous mass-delete)

12. Trade-offs & defeaters

- Defeater: "store blobs in the KV store" — value-size caps exist because big values wreck compaction; claim-check: blob here, reference there

- Defeater: "replicate via the metadata service" — data through the metadata tier melts it; clients ↔ chunk servers directly
- Defeater: "EC everything" — small-object overhead + repair amplification; hybrid by size/temperature
- Consistency: metadata strong, data immutable → the classic "eventual consistency" trap doesn't apply to committed objects

13. Pop-ups

- ? Multipart/resumable upload API? → exactly s1.09 — parts = chunks, complete = manifest commit
- ? Listing a bucket with 100M keys? → paginated ordered scan on (bucket, key); no "list all"
- ? How is this different from HDFS/GFS? → same metadata/chunk split; object stores drop POSIX semantics (no append/rename tree) to scale namespace
- **L5:** cell architecture — metadata + chunk fleet per cell; placement across FDs within AD; cross-region = async replication policy per bucket

14. Memorization card

- Metadata (strong, small, sharded) || data (immutable 64 MB chunks, dumb servers)
- Write: chunks first (idempotent by sha256 cid) → manifest commit atomic; orphans GC'd
- Placement across fault domains; heartbeat + inventory diff → repair (1-copy first); scrub for bit rot
- 3× hot/small, EC 8+4 cold/large; blobs never in the KV — claim check